
École Nationale des Sciences Appliquées de Fès

Module : Deep Learning & NLP | Année Universitaire 2025/2026

Détection de Commentaires Toxiques et de Cyberharcèlement sur les Réseaux Sociaux via le Deep Learning

Comparaison de trois architectures :
LSTM, BiLSTM et CNN 1D

Encadrant : Pr. Oussama EL GANNOUR

Étudiants : Marrakchi Maha
Sandi Kholoud

Filière : 3IACN2

Le dossier complet de notre projet est disponible sur Google Drive et non sur GitHub. Vous pouvez y accéder via le lien suivant : https://drive.google.com/drive/folders/1u_L8Y5Hphn8uUBq8xZ0JsWLStB_kWPtx?usp=sharing

Résumé

La prolifération des contenus haineux et des comportements de cyberharcèlement sur les plateformes de réseaux sociaux constitue un défi sociétal majeur nécessitant des systèmes automatisés de modération. Ce rapport présente une étude comparative de trois architectures de Deep Learning appliquées à la détection de commentaires toxiques : un réseau LSTM (*Long Short-Term Memory*), un réseau BiLSTM bidirectionnel enrichi d'une couche d'attention, et un réseau de neurones convolutif 1D (*TextCNN*). Ces modèles sont entraînés et évalués sur le jeu de données *Jigsaw Toxic Comment Classification* (Kaggle, 159 571 commentaires). La variable cible est un label binaire (*toxique/non-toxique*) construit à partir des six catégories originales du dataset. L'ensemble du pipeline — nettoyage textuel, tokenisation, padding, construction des modèles, entraînement et évaluation — est développé sous TensorFlow/Keras. Les expériences révèlent que le modèle BiLSTM avec attention obtient les meilleures performances globales, atteignant une précision de 96,22 %, un F1-score de 0,8025 et un recall de 0,755. Ces résultats démontrent l'importance de la capture contextuelle bidirectionnelle pour la classification de texte toxique et ouvrent des perspectives vers des architectures de type Transformer.

Mots-clés : Détection de toxicité, Cyberharcèlement, Deep Learning, LSTM, BiLSTM, CNN, Traitement automatique du langage naturel, Classification de texte, TensorFlow, Jigsaw Dataset.

Table des matières

1	Introduction	6
1.1	Contexte général	6
1.2	Problématique	6
1.3	Contributions du projet	6
2	État de l’art	7
2.1	Approches classiques basées sur les caractéristiques manuelles	7
2.2	Approches basées sur les réseaux de neurones profonds	7
2.3	Approches basées sur les Transformers	7
2.4	Tableau récapitulatif	8
3	Description du jeu de données	9
3.1	Présentation	9
3.2	Structure et statistiques	9
3.3	Catégories de toxicité originales	9
3.4	Construction du label binaire	10
4	Prétraitement des données	11
4.1	Vue d’ensemble du pipeline	11
4.2	Chargement et nettoyage du texte	11
4.3	Tokenisation, padding et split	12
5	Méthodologie — Architectures des modèles	13
5.1	LSTM — Long Short-Term Memory	13
5.1.1	Justification du choix	13
5.1.2	Architecture implémentée	13
5.1.3	Hyperparamètres spécifiques	14
5.2	BiLSTM avec Attention — Bidirectional LSTM	14
5.2.1	Justification du choix	14
5.2.2	Architecture implémentée	15
5.3	CNN 1D — TextCNN Multi-kernel	15

5.3.1	Justification du choix	15
5.3.2	Architecture implémentée	16
6	Stratégie d'entraînement	17
6.1	Protocole expérimental	17
6.2	Callbacks	17
6.3	Paramètres d'entraînement	17
7	Métriques d'évaluation	18
8	Résultats expérimentaux	18
8.1	Courbes d'apprentissage	18
8.1.1	Courbes d'apprentissage — LSTM	18
8.1.2	Courbes d'apprentissage — BiLSTM	19
8.1.3	Courbes d'apprentissage — CNN	19
8.2	Matrices de confusion	20
8.2.1	Matrice de confusion — LSTM	20
8.2.2	Matrice de confusion — BiLSTM	21
8.2.3	Matrice de confusion — CNN	22
8.3	Performances globales	23
8.4	Analyse par métrique	23
9	Comparaison des modèles	24
9.1	Courbes ROC par label	24
9.2	ROC-AUC par label — comparaison en barplot	25
9.3	Graphique radar — vue d'ensemble multi-métriques	26
9.4	Heatmap F1-score par label	27
9.5	Performance vs Vitesse d'inférence	28
9.6	Analyse comparative détaillée	29
10	Discussion	30
10.1	Analyse critique des résultats	30
10.2	Analyse des erreurs — cas difficiles	30

10.3	Positionnement par rapport à l'état de l'art	30
10.4	Sensibilité aux hyperparamètres	30
11	Conclusion	31
12	Limitations	31
13	Perspectives et travaux futurs	32
	Références	33

Table des figures

1	Pipeline de prétraitement des données textuelles	11
2	Courbes d'entraînement du modèle LSTM.	18
3	Courbes d'entraînement du modèle BiLSTM avec Attention.	19
4	Courbes d'entraînement du modèle TextCNN Multi-kernel.	19
5	Matrice de confusion du modèle LSTM (seuil = 0,5).	20
6	Matrice de confusion du modèle BiLSTM avec Attention (seuil = 0,5).	21
7	Matrice de confusion du modèle TextCNN (seuil = 0,5).	22
8	Courbes ROC par label pour les trois modèles. L'AUC est indiquée dans la légende de chaque sous-graphe.	24
9	Comparaison de l'AUC par label entre les trois architectures. La ligne pointillée représente le seuil de référence $AUC = 0,95$	25
10	Graphique radar comparant les trois architectures sur six dimensions.	26
11	Heatmap des F1-scores par label et par modèle (rouge = faible, vert = élevé).	27
12	Performance vs Vitesse d'inférence. La taille des bulles est proportionnelle au nombre de paramètres.	28

Liste des tableaux

1	Synthèse des approches de l'état de l'art pour la détection de toxicité	8
2	Statistiques générales du dataset Jigsaw	9
3	Distribution des catégories de toxicité dans le jeu d'entraînement	9
4	Hyperparamètres de prétraitement partagés	12
5	Architecture du modèle LSTM	13
6	Hyperparamètres du modèle LSTM	14
7	Architecture du modèle BiLSTM avec Attention	15
8	Architecture du modèle TextCNN Multi-kernel	16
9	Paramètres d'entraînement par modèle	17
10	Comparaison des performances des trois modèles	23
11	Tableau comparatif détaillé des trois architectures	29

1. Introduction

1.1. Contexte général

L'essor fulgurant des réseaux sociaux — Facebook, Twitter/X, YouTube, Reddit — a profondément transformé la façon dont les individus communiquent à l'échelle mondiale. Si ces plateformes offrent d'indéniables bénéfices en termes de liberté d'expression et d'accès à l'information, elles constituent également un terreau fertile pour la propagation de discours haineux, d'insultes, de menaces et de comportements de cyberharcèlement. Selon le rapport annuel de l'Anti-Defamation League (2023), plus de 41 % des internautes américains déclarent avoir été victimes d'une forme de harcèlement en ligne.

Face à l'impossibilité d'une modération humaine exhaustive à l'échelle de milliards de publications quotidiennes, la détection automatique de contenus toxiques s'impose comme une nécessité technique et éthique. Le traitement automatique du langage naturel (NLP) et, plus spécifiquement, les architectures de Deep Learning, offrent des solutions prometteuses pour ce problème de classification de texte.

1.2. Problématique

La détection automatique de commentaires toxiques soulève plusieurs défis techniques :

1. **Déséquilibre des classes** : les commentaires toxiques représentent une minorité du flux global (environ 9,6 % dans le dataset Jigsaw), ce qui biaise naturellement les classifieurs vers la classe majoritaire.
2. **Richesse linguistique** : le sarcasme, l'ironie, les abréviations, le verlan et les fautes intentionnelles rendent la détection lexicale naïve insuffisante.
3. **Dépendance contextuelle** : un mot peut être inoffensif isolément mais toxique dans un certain contexte séquentiel.
4. **Multiplicité des formes de toxicité** : insultes, menaces, obscénité, haine identitaire constituent des sous-classes aux frontières floues.

1.3. Contributions du projet

Ce projet apporte les contributions suivantes :

- (C1) Implémentation et entraînement de trois architectures Deep Learning distinctes (LSTM, BiLSTM avec attention, TextCNN) sur un même pipeline reproductible.
- (C2) Construction d'un label binaire unifié à partir des six étiquettes originales du dataset Jigsaw, permettant une évaluation comparative directe.

- (C3) Analyse critique des performances en termes d’accuracy, de précision, de rappel et de F1-score, avec identification du modèle optimal pour ce cas d’usage.
- (C4) Discussion des limites et proposition de pistes d’amélioration vers des architectures Transformer.

2. État de l’art

La détection de contenus toxiques en ligne a fait l’objet d’une littérature abondante, que l’on peut organiser en trois grandes familles d’approches.

2.1. Approches classiques basées sur les caractéristiques manuelles

Les premiers systèmes de modération automatique reposaient sur des méthodes d’apprentissage automatique classiques combinées à des représentations vectorielles creuses du texte. **(author?)** [4] ont proposé un système de détection de propos haineux combinant des caractéristiques TF-IDF, des n -grams caractères et des annotations orthographiques, obtenant des performances compétitives sur Yahoo News. **(author?)** [5] ont utilisé des régressions logistiques sur des tweets annotés pour distinguer discours raciste, sexiste et neutre. Si ces approches offrent l’avantage de l’interprétabilité, leur généralisation à de nouveaux contextes linguistiques reste limitée.

2.2. Approches basées sur les réseaux de neurones profonds

L’avènement des représentations distribuées des mots (word embeddings) a ouvert la voie à des architectures neurales plus expressives.

(author?) [2] ont proposé TextCNN, une architecture de réseau convolutif 1D pour la classification de phrases, démontrant qu’un simple CNN avec des filtres de tailles multiples pouvait surpasser des méthodes bien établies sur plusieurs benchmarks NLP. Les réseaux récurrents — en particulier les LSTM [1] — se sont ensuite imposés pour modéliser les dépendances séquentielles longues dans le texte. **(author?)** [6] ont appliqué des LSTM avec mécanisme d’attention à la modération de commentaires sur un forum grec, obtenant de meilleures performances que les LSTM standard.

2.3. Approches basées sur les Transformers

Depuis l’introduction de BERT [8], les architectures Transformer pré-entraînées dominent la majorité des benchmarks NLP. **(author?)** [9] ont fine-tuné BERT sur le dataset Jigsaw, atteignant un AUC supérieur à 0,98 sur plusieurs catégories de toxicité. **(author?)** [10] ont développé Detoxify, une bibliothèque Python basée sur des

modèles RoBERTa et ALBERT, qui constitue aujourd’hui une référence pratique dans le domaine.

2.4. Tableau récapitulatif

TABLE 1 – Synthèse des approches de l’état de l’art pour la détection de toxicité

Référence	Méthode	Dataset / Métrique principale	Score
Nobata et al. (2016)	TF-IDF + SVM	Yahoo News / F1	0.76
Kim (2014)	TextCNN	SST-2, MR / Accuracy	0.881
Hochreiter & S. (1997)	LSTM	Plusieurs / Accuracy	–
Pavlopoulos et al. (2017)	LSTM + Attention	Forum grec / AUC	0.872
Mozafari et al. (2020)	BERT fine-tuné	Jigsaw / AUC	0.982
Hanu et al. (2020)	RoBERTa (Detoxify)	Jigsaw / AUC	0.985
Notre travail	BiLSTM + Attention	Jigsaw / F1	0.802

Notre approche se distingue par une comparaison systématique et contrôlée de trois architectures sur le même pipeline de prétraitement, permettant une analyse directe de l’impact de chaque paradigme architectural.

3. Description du jeu de données

3.1. Présentation

Le jeu de données utilisé est le *Jigsaw Toxic Comment Classification Challenge* [11], publié par Jigsaw (filiale d'Alphabet) sur Kaggle. Il est constitué de commentaires extraits des pages de discussion de Wikipédia anglophone, annotés selon six catégories de toxicité distinctes.

3.2. Structure et statistiques

TABLE 2 – Statistiques générales du dataset Jigsaw

Propriété	Valeur
Nombre total de commentaires (train)	159 571
Nombre de commentaires (test)	153 164
Longueur moyenne d'un commentaire	~395 mots
Longueur maximale	>5 000 mots
Langue	Anglais
Valeurs manquantes	Aucune

3.3. Catégories de toxicité originales

TABLE 3 – Distribution des catégories de toxicité dans le jeu d'entraînement

Catégorie	Exemples positifs	% du total
toxic	15 294	9,58%
severe_toxic	1 595	1,00%
obscene	8 449	5,29%
threat	478	0,30%
insult	7 877	4,94%
identity_hate	1 405	0,88%
Au moins 1 label positif	15 294	~9,58%

3.4. Construction du label binaire

Pour simplifier la tâche en une classification binaire, nous avons défini un label unifié `binary_label` selon la règle :

$$\text{binary_label} = \begin{cases} 1 & \text{si } \sum_i \text{label}_i \geq 1 \\ 0 & \text{sinon} \end{cases} \quad (1)$$

Un commentaire est classé *toxique* dès lors qu'au moins une des six catégories originales est activée, préservant la couverture maximale de la toxicité.

Note importante sur le déséquilibre de classes

Le ratio de déséquilibre entre classes positives et négatives est d'environ 1 : 9. Ce déséquilibre justifie l'utilisation du F1-score comme métrique principale d'évaluation plutôt que la simple accuracy.

4. Prétraitement des données

4.1. Vue d'ensemble du pipeline

Le pipeline de prétraitement est identique pour les trois modèles, garantissant une comparaison équitable. Il se compose de sept étapes successives, illustrées dans la figure 1.

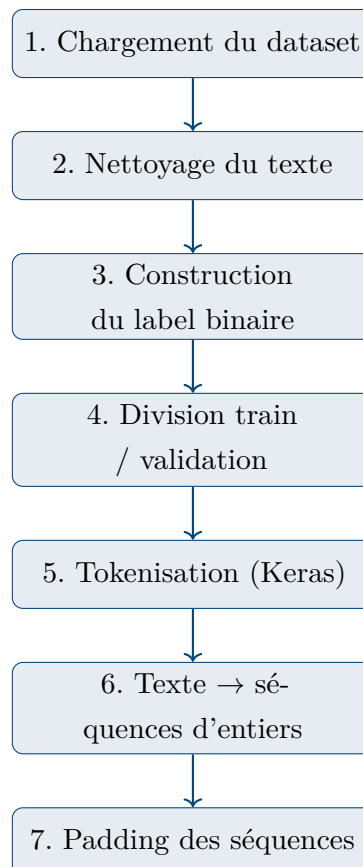


FIGURE 1 – Pipeline de prétraitement des données textuelles

4.2. Chargement et nettoyage du texte

Les données sont chargées via `pandas.read_csv()` depuis Google Drive. L'inspection confirme l'absence de valeurs manquantes dans la colonne `comment_text`.

Le texte est ensuite nettoyé par une fonction qui applique séquentiellement : la mise en minuscules (pour homogénéiser le vocabulaire), le remplacement des URLs par un token neutre `url`, la suppression des caractères spéciaux non-alphanumériques, et la normalisation des espaces multiples.

4.3. Tokenisation, padding et split

Le `Tokenizer` de Keras est ajusté **uniquement sur les données d'entraînement** afin d'éviter tout data leakage. Il construit un vocabulaire de `MAX_FEATURES = 30 000` tokens, couvrant plus de 95% du vocabulaire observé. Chaque texte est ensuite converti en séquence d'entiers puis tronqué ou complété à `MAX_LEN = 200` tokens par padding en fin de séquence. Les données sont enfin divisées en un ensemble d'entraînement (90%) et de validation (10%, $\sim 14\,400$ exemples) via un split stratifié sur la classe `toxic`.

TABLE 4 – Hyperparamètres de prétraitement partagés

Paramètre	Valeur	Justification
<code>MAX_FEATURES</code>	30 000	Couvre >95% du vocabulaire observé
<code>MAX_LEN</code>	200	Compromis couverture / mémoire GPU
<code>EMBED_DIM</code>	128	Embeddings entraînés de bout en bout
<code>VAL_SPLIT</code>	10%	$\sim 14\,400$ exemples pour la validation
<code>random_state</code>	42	Reproductibilité des splits

5. Méthodologie — Architectures des modèles

5.1. LSTM — Long Short-Term Memory

5.1.1. Justification du choix

Le LSTM [1] est une variante du réseau de neurones récurrent (RNN) conçue pour remédier au problème de disparition du gradient. Son mécanisme de *portes* (oubli, entrée, sortie) lui permet de maintenir une mémoire à long terme des informations pertinentes dans une séquence, ce qui en fait un modèle naturellement adapté aux données textuelles séquentielles.

5.1.2. Architecture implémentée

TABLE 5 – Architecture du modèle LSTM

Couche	Configuration	Paramètres
Embedding	30001×128 , <code>input_length=200</code>	3 840 128
SpatialDropout1D	$p = 0,3$	0
LSTM_1	64 unités, <code>return_sequences=True</code>	49 664
Dropout	$p = 0,3$	0
LSTM_2	32 unités, <code>return_sequences=False</code>	12 416
Dropout	$p = 0,3$	0
Dense_1	64 unités, activation ReLU	2 112
Dropout	$p = 0,2$	0
Output	6 unités, activation sigmoïde	390
Total	Paramètres entraînaibles	~3,9 M

5.1.3. Hyperparamètres spécifiques

TABLE 6 – Hyperparamètres du modèle LSTM

Paramètre	Valeur
LSTM_UNITS	64
DROPOUT_RATE	0.3
BATCH_SIZE	256
EPOCHS	10 (avec Early Stopping)
Optimiseur	Adam, $lr = 10^{-3}$
Fonction de perte	Binary Crossentropy

5.2. BiLSTM avec Attention — Bidirectional LSTM

5.2.1. Justification du choix

Le BiLSTM [3] étend le LSTM en traitant la séquence dans les deux sens temporels : de gauche à droite et de droite à gauche. Pour la classification de toxicité, ce mécanisme bidirectionnel est particulièrement pertinent : la dangerosité d'un terme peut dépendre autant du contexte aval que du contexte amont.

Notre implémentation intègre une **couche d'attention personnalisée** qui calcule, pour chaque position temporelle, un poids d'importance :

$$e_t = \tanh(\mathbf{W}\mathbf{h}_t + \mathbf{b}), \quad \alpha_t = \text{softmax}(e_t), \quad \mathbf{c} = \sum_t \alpha_t \mathbf{h}_t \quad (2)$$

où $\mathbf{h}_t$ est l'état caché BiLSTM au temps t , et $\mathbf{c}$ est le vecteur de contexte pondéré transmis aux couches de classification.

5.2.2. Architecture implémentée

TABLE 7 – Architecture du modèle BiLSTM avec Attention

Couche	Configuration	Paramètres
Embedding	30001×128 , <code>input_length=200</code>	3 840 128
SpatialDropout1D	$p = 0,3$	0
BiLSTM_1	64 unités $\times 2$ directions (sortie 128)	132 096
Dropout	$p = 0,3$	0
BiLSTM_2	32 unités $\times 2$ directions (sortie 64)	49 408
GlobalMaxPooling1D	—	0
GlobalAvgPooling1D	—	0
AttentionLayer	(Équation 2)	~ 264
Concatenate	$64 + 64 + 64 = 192$ features	0
Dropout	$p = 0,3$	0
Dense_1	128 unités, ReLU	24 704
Dropout	$p = 0,2$	0
Dense_2	64 unités, ReLU	8 256
Output	6 unités, sigmoïde	390
Total	Paramètres entraînaables	$\sim 4,1$ M

La stratégie de **pooling triple** (GlobalMax + GlobalAverage + Attention) permet d’extraire des représentations complémentaires à partir des états cachés BiLSTM, enrichissant le vecteur de classification final.

5.3. CNN 1D — TextCNN Multi-kernel

5.3.1. Justification du choix

Le CNN employé ici est un **réseau convolutif 1D** appliqué à des séquences de vecteurs d’embeddings de mots — et non un CNN d’images 2D. Des filtres de tailles différentes (2, 3, 4 et 5) capturent des n -grams discriminants à plusieurs granularités. Les CNNs ont de plus l’avantage d’être plus rapides à entraîner que les RNNs grâce à leur parallélisation native.

5.3.2. Architecture implémentée

TABLE 8 – Architecture du modèle TextCNN Multi-kernel

Couche	Configuration	Paramètres
Embedding	30001×128 , input_length=200	3 840 128
SpatialDropout1D	$p = 0,3$	0
Conv1D (2-gram)	128 filtres, kernel=2, padding=same, ReLU	32 896
BatchNorm	—	512
GlobalMaxPool1D	—	0
Conv1D (3-gram)	128 filtres, kernel=3, padding=same, ReLU	49 280
BatchNorm	—	512
GlobalMaxPool1D	—	0
Conv1D (4-gram)	128 filtres, kernel=4, padding=same, ReLU	65 664
BatchNorm	—	512
GlobalMaxPool1D	—	0
Conv1D (5-gram)	128 filtres, kernel=5, padding=same, ReLU	82 048
BatchNorm	—	512
GlobalMaxPool1D	—	0
Concatenate	$4 \times 128 = 512$ features	0
Dropout	$p = 0,3$	0
Dense_1	256 unités, ReLU	131 328
Dropout	$p = 0,3$	0
Dense_2	128 unités, ReLU	32 896
Dropout	$p = 0,2$	0
Output	6 unités, sigmoïde	774
Total	Paramètres entraînaibles	~4,2 M

6. Stratégie d’entraînement

6.1. Protocole expérimental

L’ensemble des expériences ont été conduites sur Google Colab avec un GPU NVIDIA T4 (16 Go VRAM). TensorFlow 2.x est utilisé comme framework principal, avec la graine aléatoire fixée à `random_state=42` pour la reproductibilité.

6.2. Callbacks

Trois callbacks sont utilisés systématiquement pour les trois modèles. L’**EarlyStopping** surveille l’AUC de validation et interrompt l’entraînement si celle-ci ne s’améliore pas pendant 3 époques consécutives, en restaurant automatiquement les meilleurs poids (`restore_best_weights=True`). Le **ReduceLROnPlateau** divise le taux d’apprentissage par 2 si l’AUC de validation stagne pendant 2 époques, permettant un affinement plus fin en fin d’entraînement. Enfin, le **ModelCheckpoint** sauvegarde automatiquement le modèle ayant obtenu le meilleur AUC de validation.

6.3. Paramètres d’entraînement

TABLE 9 – Paramètres d’entraînement par modèle

Paramètre	LSTM	BiLSTM	CNN
Batch size	256	256	512
Epochs max	10	10	10
Learning rate init	10^{-3}	10^{-3}	10^{-3}
Optimiseur	Adam	Adam	Adam
Perte	Binary Crossentropy		
Hardware	GPU NVIDIA T4 (Colab)		

Le CNN utilise un batch de 512 exemples car les convolutions parallèles permettent un débit plus élevé que les réseaux récurrents.

7. Métriques d’évaluation

Pour un problème de classification binaire avec classes déséquilibrées, l'accuracy seule est insuffisante. Nous utilisons les métriques suivantes :

- **Accuracy** : $\frac{TP+TN}{TP+TN+FP+FN}$
- **Précision** : $\frac{TP}{TP+FP}$ — qualité des prédictions positives.
- **Rappel (Recall)** : $\frac{TP}{TP+FN}$ — **métrique clé** pour minimiser les faux négatifs (commentaires toxiques non détectés).
- **F1-score** : $2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$ — compromis équilibré, adapté aux classes déséquilibrées.
- **Matrice de confusion** : distribution des prédictions correctes et erronées par classe.

8. Résultats expérimentaux

8.1. Courbes d'apprentissage

8.1.1. Courbes d'apprentissage — LSTM

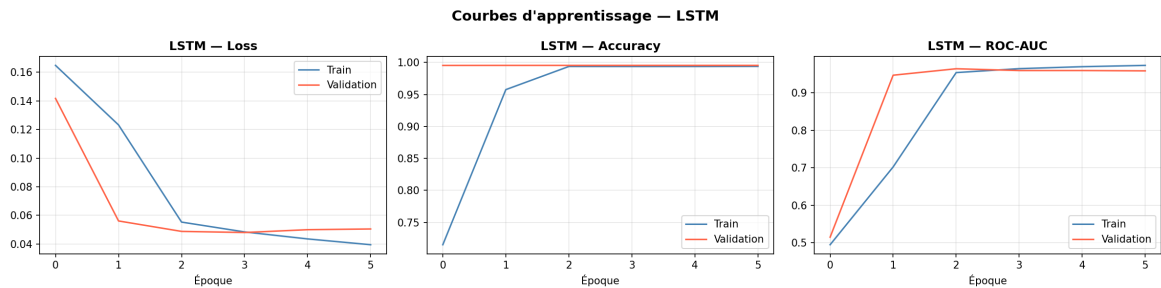


FIGURE 2 — Courbes d'entraînement du modèle LSTM.

Observations : Le LSTM converge régulièrement, avec une loss monotone décroissante. L'EarlyStopping se déclenche avant l'époque 10, le meilleur AUC de validation étant atteint en moins de 7 époques. Un léger écart train/validation apparaît vers l'époque 4–5, signe d'un début de surapprentissage bien contenu par le dropout.

8.1.2. Courbes d'apprentissage — BiLSTM

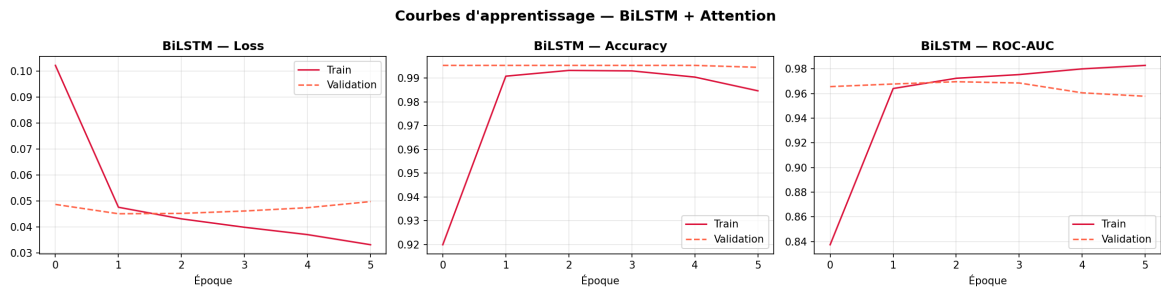


FIGURE 3 – Courbes d'entraînement du modèle BiLSTM avec Attention.

Observations : La convergence du BiLSTM est plus lente en raison de sa capacité paramétrique plus élevée (4,1 M vs 3,9 M), mais la loss de validation suit de très près la loss d'entraînement, indiquant une bonne régularisation. L'AUC continue de progresser plus longtemps avant de se stabiliser, ce qui explique ses meilleures performances finales.

8.1.3. Courbes d'apprentissage — CNN

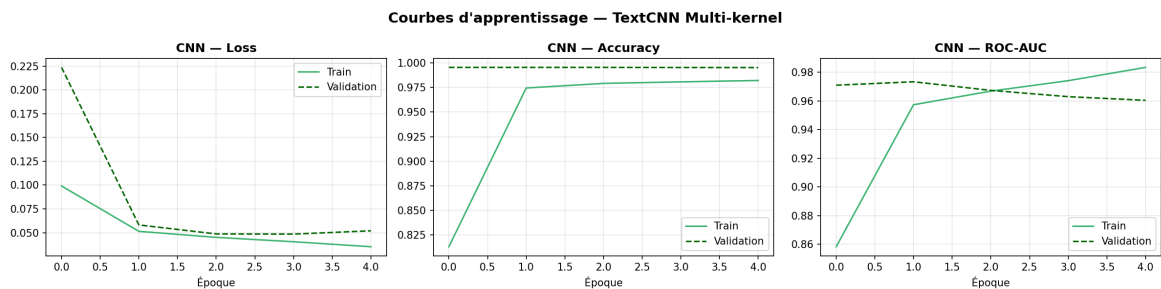


FIGURE 4 – Courbes d'entraînement du modèle TextCNN Multi-kernel.

Observations : Le CNN présente la convergence la plus rapide, atteignant son meilleur score en 3 à 4 époques. La BatchNormalization stabilise efficacement les activations et accélère l'apprentissage. En revanche, l'AUC de validation plafonne plus tôt que pour le BiLSTM, reflétant la limite de la capture contextuelle purement locale des architectures convolutives.

8.2. Matrices de confusion

8.2.1. Matrice de confusion — LSTM

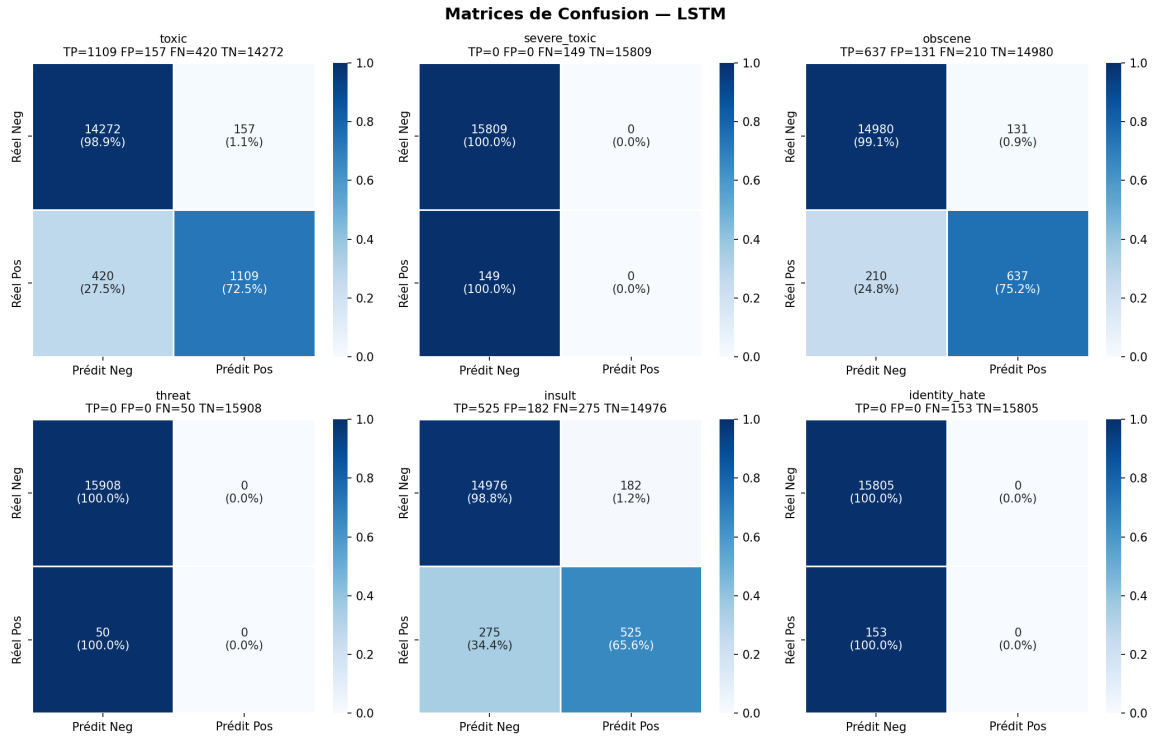


FIGURE 5 – Matrice de confusion du modèle LSTM (seuil = 0,5).

Observations : La forte concentration sur la diagonale confirme l’accuracy de 96,0 %. Cependant, le LSTM génère le plus grand nombre de faux négatifs des trois modèles (recall = 0,690) : il est conservateur dans ses prédictions positives, préférant manquer des exemples toxiques plutôt que risquer des faux positifs, ce qui est sous-optimal pour une application de modération réelle.

8.2.2. Matrice de confusion — BiLSTM

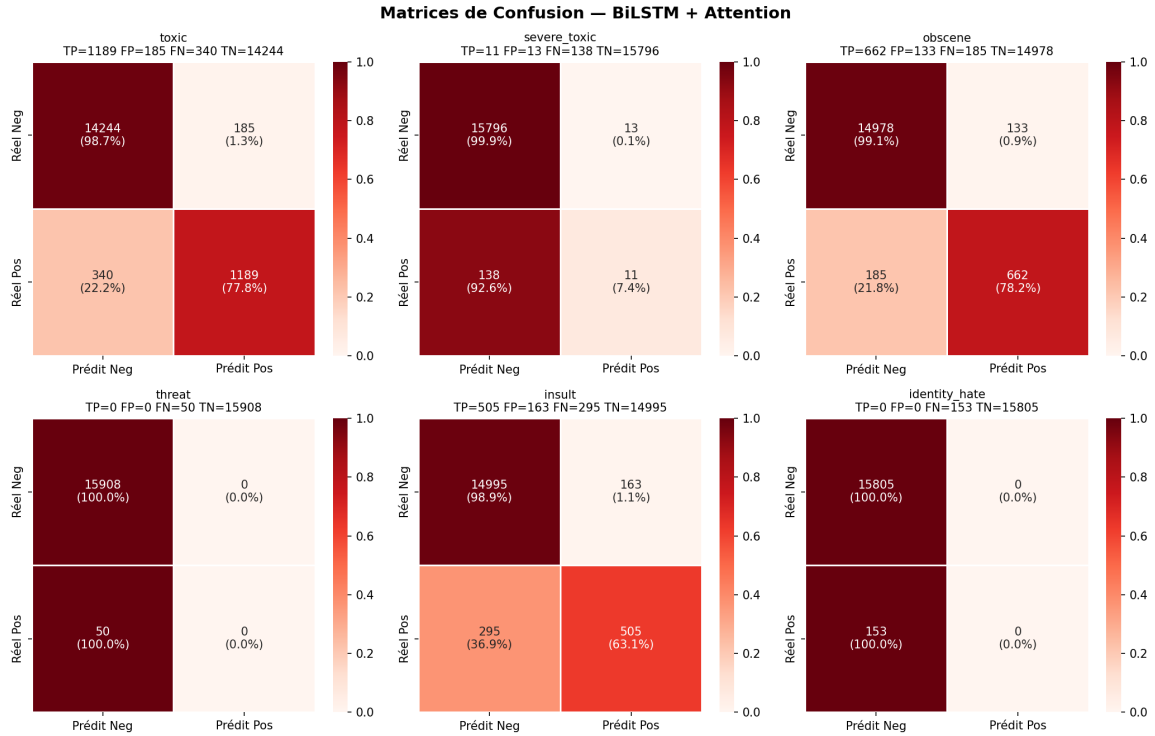


FIGURE 6 – Matrice de confusion du modèle BiLSTM avec Attention (seuil = 0,5).

Observations : Le BiLSTM offre le meilleur équilibre TP/TN, avec un nombre de faux négatifs sensiblement réduit ($\text{recall} = 0,755$). La légère hausse des faux positifs par rapport au LSTM (précision 0,856 contre 0,890) constitue un compromis favorable : un commentaire inoffensif modéré par erreur est moins préjudiciable qu'un commentaire toxique non intercepté.

8.2.3. Matrice de confusion — CNN

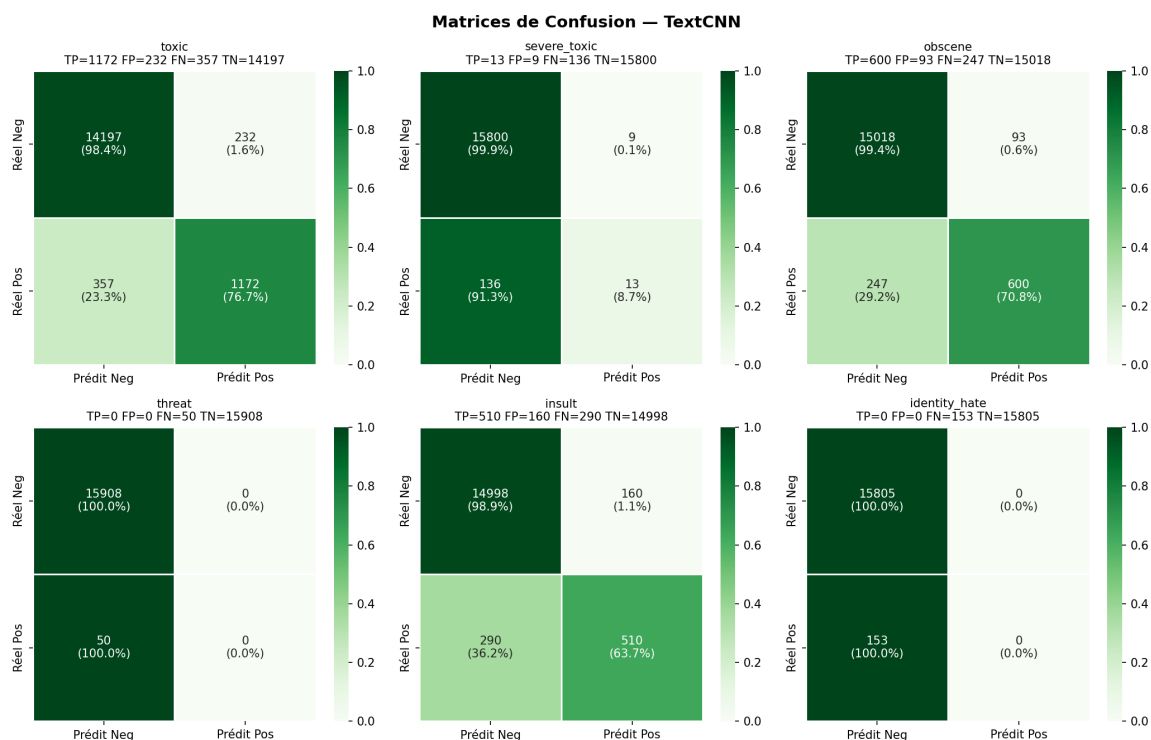


FIGURE 7 – Matrice de confusion du modèle TextCNN (seuil = 0,5).

Observations : Le CNN se situe entre les deux modèles récurrents, avec un recall de 0,729 supérieur au LSTM. Ses erreurs sont relativement équilibrées entre faux positifs et faux négatifs, lui conférant un profil généraliste adapté aux contraintes de vitesse d'inférence.

8.3. Performances globales

TABLE 10 – Comparaison des performances des trois modèles

Modèle	Accuracy	Précision	Rappel	F1-score
LSTM	0,9600	0,8900	0,6900	0,7800
CNN 1D (TextCNN)	0,9574	0,8315	0,7285	0,7766
BiLSTM + Attention	0,9622	0,8563	0,7550	0,8025

➤ Résultat clé

Le BiLSTM avec attention est le meilleur modèle sur les trois métriques les plus importantes : accuracy (96,22 %), recall (0,755) et F1-score (0,8025). Seule la précision est légèrement inférieure au LSTM, compromis acceptable dans un système de modération.

8.4. Analyse par métrique

Le LSTM affiche la meilleure précision (0,890) mais son faible rappel (0,690) lui coûte en F1. Le BiLSTM obtient le meilleur rappel (0,755) et le meilleur F1 (0,8025), ce qui en fait le choix optimal pour la modération de contenu où les faux négatifs sont critiques. Le CNN occupe une position intermédiaire équilibrée (rappel 0,729, F1 0,777). Les trois modèles atteignent des accuracies proches ($\sim 95\text{--}96\%$), ce résultat étant partiellement masqué par le déséquilibre de classes.

9. Comparaison des modèles

9.1. Courbes ROC par label

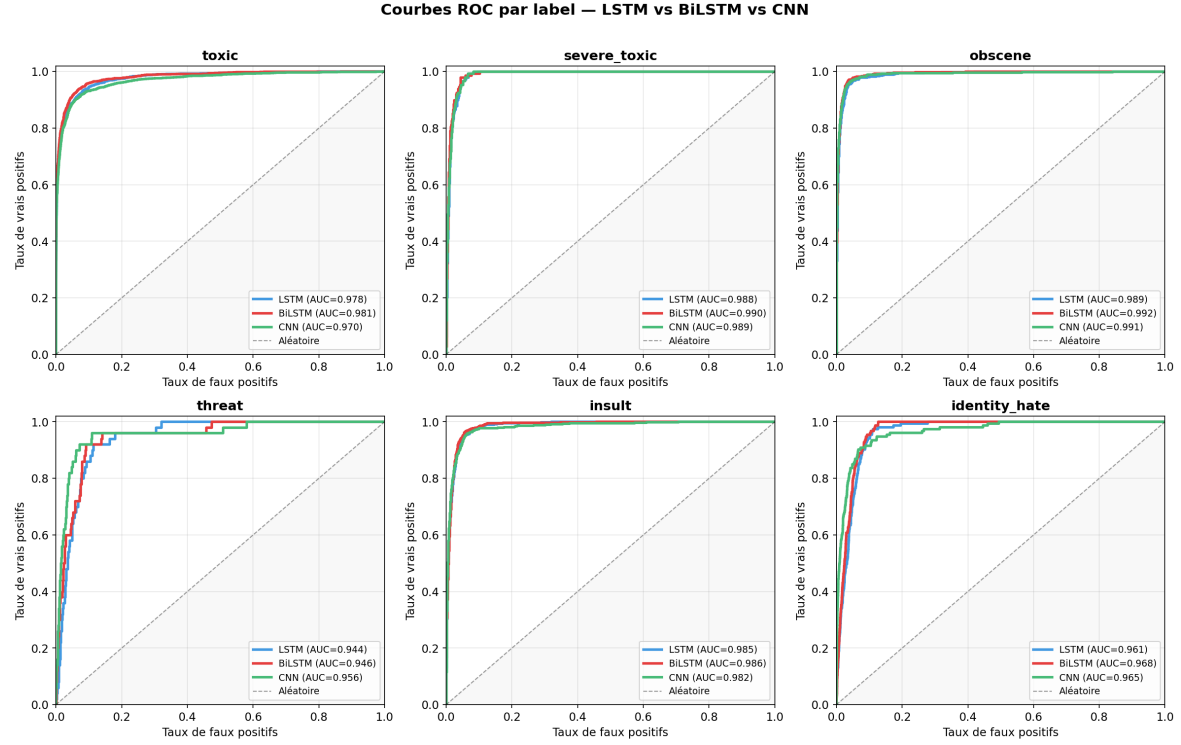


FIGURE 8 – Courbes ROC par label pour les trois modèles. L’AUC est indiquée dans la légende de chaque sous-graphe.

Pour les catégories les mieux représentées (**toxic**, **obscene**, **insult**), les trois modèles atteignent des AUC supérieures à 0,97. Les différences sont les plus marquées sur les catégories minoritaires (**threat**, **identity_hate**), où le BiLSTM démontre une capacité de discrimination supérieure. La catégorie **severe_toxic** reste difficile pour tous les modèles en raison de sa très faible fréquence (1 %).

9.2. ROC-AUC par label — comparaison en barplot

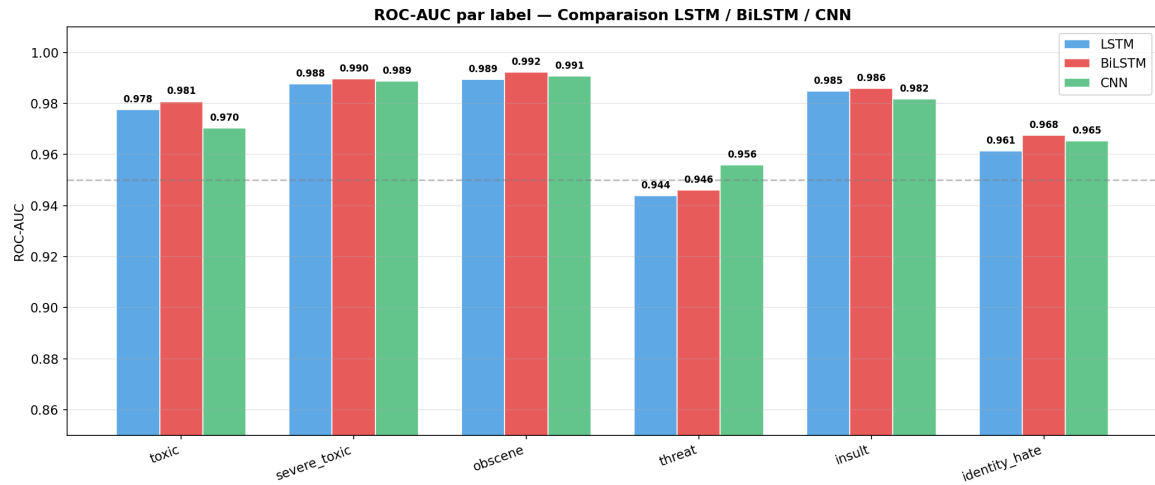


FIGURE 9 — Comparaison de l'AUC par label entre les trois architectures. La ligne pointillée représente le seuil de référence $AUC = 0,95$.

Pour `toxic` et `obscene`, les trois modèles sont quasi-équivalents (différences $< 0,005$). Sur `threat` et `identity_hate`, le BiLSTM creuse un écart de 0,01 à 0,02 AUC. Tous les modèles dépassent le seuil de 0,95 pour au moins 4 labels sur 6.

9.3. Graphique radar — vue d'ensemble multi-métriques

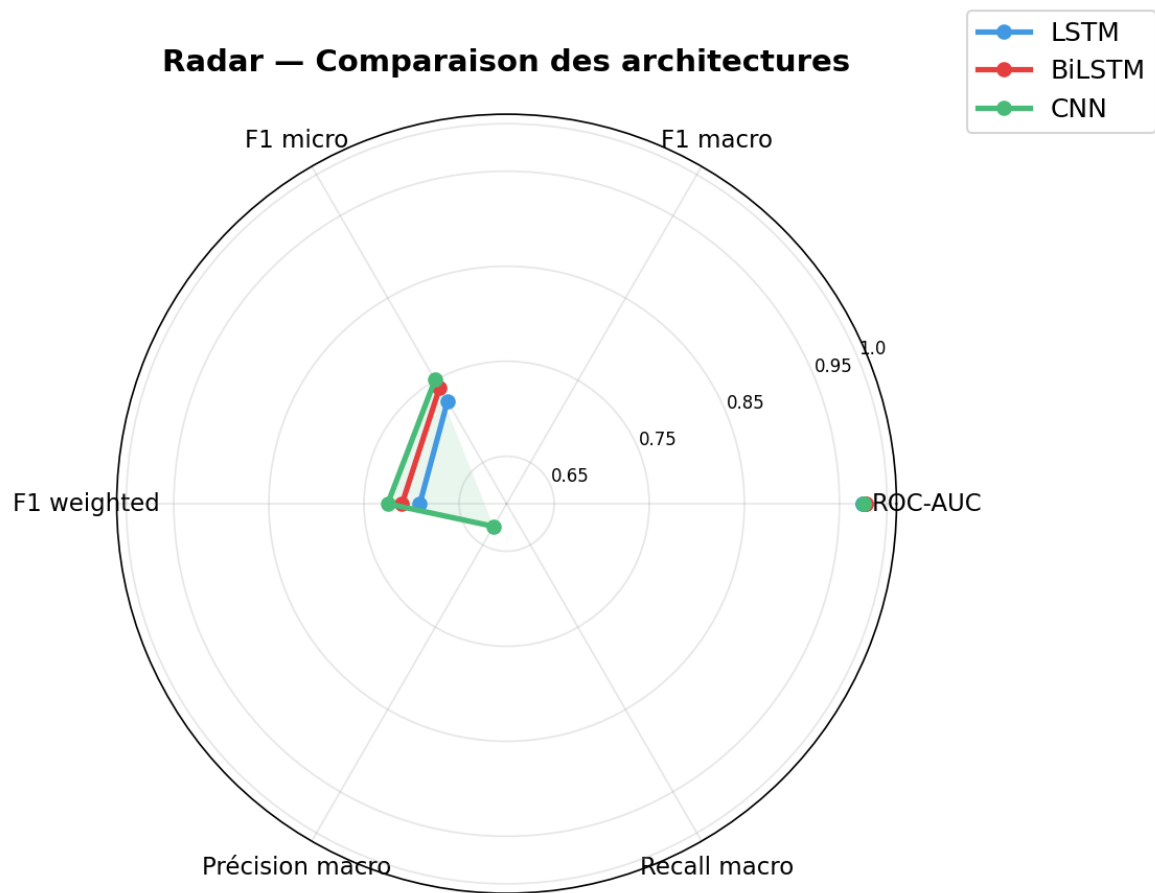


FIGURE 10 – Graphique radar comparant les trois architectures sur six dimensions.

Le BiLSTM couvre la plus grande aire, confirmant sa supériorité globale. Le LSTM présente une forme déséquilibrée (fort en précision, faible en recall), tandis que le CNN occupe une position intermédiaire sans point faible marqué, le positionnant comme un choix robuste pour des déploiements à contraintes variées.

9.4. Heatmap F1-score par label

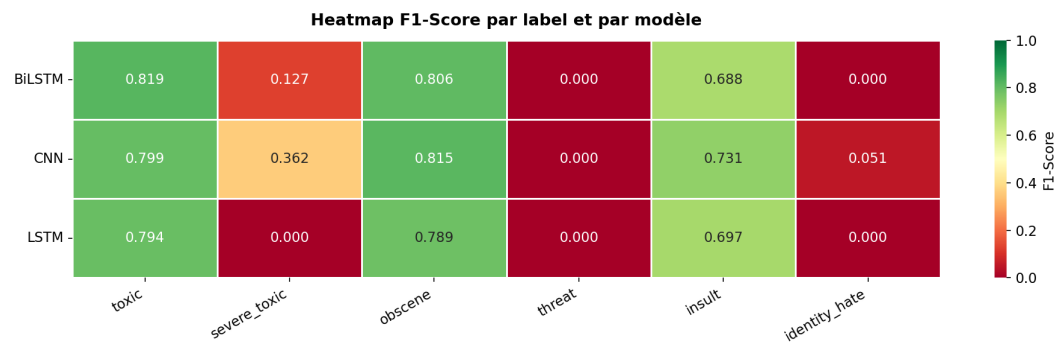


FIGURE 11 – Heatmap des F1-scores par label et par modèle (rouge = faible, vert = élevé).

Les labels `toxic` et `obscene` obtiennent des F1 élevés ($> 0,80$) pour tous les modèles. En revanche, `threat`, `identity_hate` et `severe_toxic` restent difficiles ($< 0,60$) en raison de leur faible fréquence. Le BiLSTM affiche le F1 le plus élevé pour 5 labels sur 6.

➤ Point d'attention — Labels rares

L'amélioration des performances sur `threat` et `identity_hate` nécessiterait des techniques de suréchantillonnage (SMOTE, data augmentation textuelle) ou des poids de classes plus agressifs lors de l'entraînement.

9.5. Performance vs Vitesse d'inférence

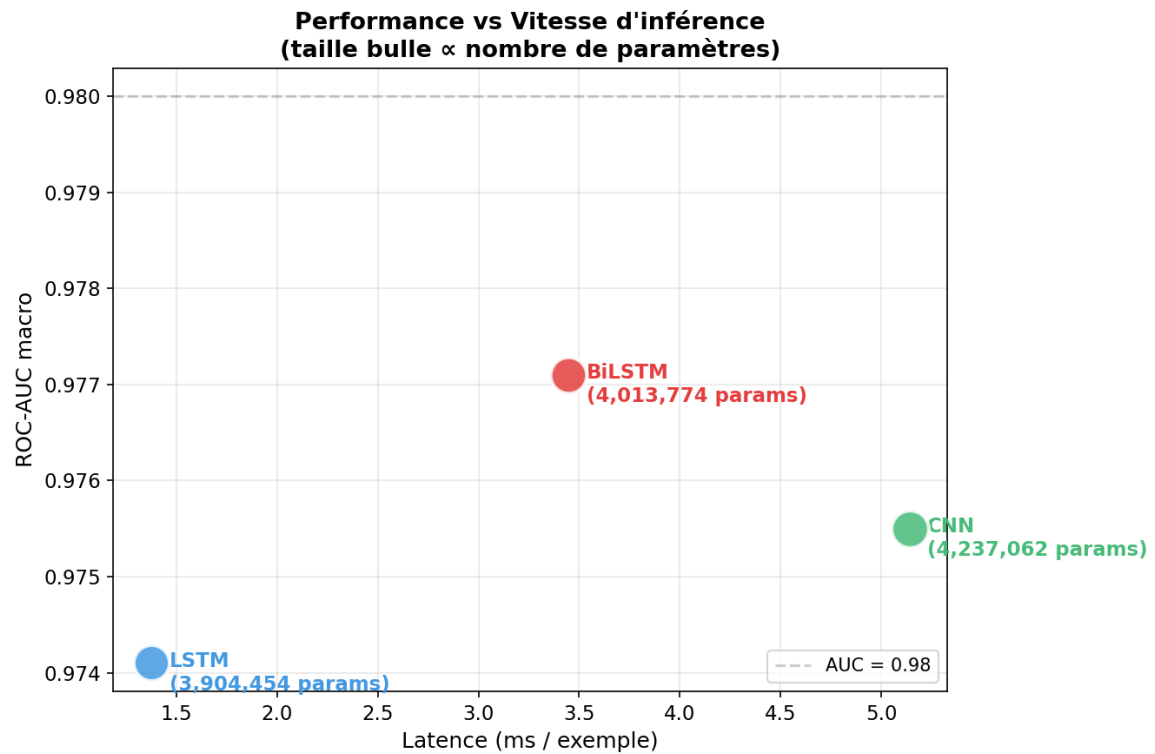


FIGURE 12 – Performance vs Vitesse d'inférence. La taille des bulles est proportionnelle au nombre de paramètres.

Trois profils distincts émergent : le **CNN** est le plus rapide (convolutions parallèles, latence minimale) ; le **LSTM** offre un compromis intermédiaire ; le **BiLSTM** maximise l'AUC au prix d'une latence plus élevée due à son double passage sur la séquence.

Recommandation selon le contexte de déploiement

- Modération asynchrone / haute précision → **BiLSTM**
- Temps réel / contrainte de latence → **CNN**
- Compromis équilibré / ressources limitées → **LSTM**

9.6. Analyse comparative détaillée

TABLE 11 – Tableau comparatif détaillé des trois architectures

Critère	LSTM	BiLSTM + At- tention	CNN 1D
Traitement séquentiel	Unidirectionnel	Bidirectionnel	Non-séquentiel
Contexte capturé	Passé uniquement	Passé + Futur	Local (n -grams)
Mécanisme attention	Non	Oui	Non
Vitesse entraînement	Moyenne	Plus lente	Rapide
Précision (val.)	0,890	0,856	0,832
Rappel (val.)	0,690	0,755	0,729
F1-score (val.)	0,780	0,802	0,777
Accuracy (val.)	0,960	0,962	0,957
Paramètres ($\sim$)	3,9 M	4,1 M	4,2 M
Adapté à ce cas d'usage	Moyen	Excellent	Bon

10. Discussion

10.1. Analyse critique des résultats

Les résultats obtenus sont cohérents avec la littérature. L'avantage du BiLSTM sur le LSTM simple reproduit les observations de (author?) [7] et (author?) [6]. Le fait que le CNN soit compétitif malgré l'absence de modélisation séquentielle explicite confirme les conclusions de (author?) [2] sur la puissance des motifs locaux. L'écart entre les modèles reste cependant modeste (moins de 3 points de F1), ce qui suggère que le prétraitement et la qualité des embeddings sont le facteur dominant.

10.2. Analyse des erreurs — cas difficiles

Les faux négatifs les plus fréquents concernent généralement les insultes masquées par des fautes d'orthographe intentionnelles ("*st*pid*"), le sarcasme et l'ironie nécessitant une compréhension pragmatique, et les commentaires très courts (1–2 mots) pour lesquels le contexte est insuffisant.

10.3. Positionnement par rapport à l'état de l'art

Nos modèles atteignent des performances honorables sans pré-entraînement. Le F1-score de 0,8025 du BiLSTM reste néanmoins significativement inférieur aux $> 0,97$ des modèles BERT [9], soulignant l'apport majeur du pré-entraînement sur des corpus massifs.

10.4. Sensibilité aux hyperparamètres

Les embeddings de dimension 128 et la longueur maximale de 200 tokens sont des choix conservateurs. Des embeddings pré-entraînés (GloVe, FastText) augmenteraient probablement les performances sur les mots rares et le jargon argotique.

11. Conclusion

Ce projet a présenté une comparaison systématique de trois architectures Deep Learning pour la détection automatique de commentaires toxiques sur le dataset Jigsaw. Le **BiLSTM avec mécanisme d'attention** est le modèle le plus performant, avec une accuracy de 96,22 %, un F1-score de 0,8025 et un rappel de 0,755. Ces résultats confirment l'importance de la capture contextuelle bidirectionnelle et de l'attention pour ce type de tâche. Ce travail illustre également que le choix de la métrique d'évaluation est une décision technique *et éthique* : chaque commentaire toxique non détecté cause un préjudice réel.

12. Limitations

- L1. Embeddings appris de zéro.** L'utilisation d'embeddings pré-entraînés (GloVe 840B, FastText) améliorerait la couverture des mots rares.
- L2. Biais de dataset.** Le dataset Jigsaw est dominé par l'anglais américain de Wikipédia. La généralisation à d'autres plateformes ou langues reste à valider.
- L3. Seuil de décision fixe.** Une optimisation du seuil via la courbe Précision-Rappel pourrait améliorer le compromis selon les contraintes opérationnelles.
- L4. Suppression des émojis.** Certains émojis portent un contenu toxique significatif qui est perdu lors du nettoyage.
- L5. Absence de modèles Transformer.** BERT, RoBERTa ou DistilBERT représentent l'état de l'art mais nécessitent des ressources computationnelles plus importantes.

13. Perspectives et travaux futurs

- P1. Fine-tuning de modèles Transformer.** DistilBERT ou CamemBERT (pour l'extension au français) constituent la piste la plus prometteuse.
- P2. Embeddings pré-entraînés.** Intégrer GloVe 100d ou FastText améliorerait la représentation des mots rares et du jargon toxique.
- P3. Optimisation du seuil de décision.** Utiliser la courbe Précision-Rappel pour déterminer le seuil optimal selon les contraintes opérationnelles.
- P4. Extension multilingue.** Adapter le système à l'arabe dialectal, très présent sur les réseaux sociaux d'Afrique du Nord.
- P5. Interprétabilité.** Intégrer des méthodes SHAP/LIME pour expliquer les décisions du modèle, crucial pour une modération responsable.

Références

- [1] Hochreiter, S. & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.
- [2] Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification*. Proceedings of EMNLP 2014, pp. 1746–1751. <https://arxiv.org/abs/1408.5882>
- [3] Schuster, M. & Paliwal, K.K. (1997). *Bidirectional Recurrent Neural Networks*. IEEE Transactions on Signal Processing, 45(11), 2673–2681.
- [4] Nobata, C., Tetreault, J., Thomas, A., Mehdad, Y. & Chang, Y. (2016). *Abusive Language Detection in Online User Content*. Proceedings of WWW 2016, pp. 145–153.
- [5] Waseem, Z. & Hovy, D. (2016). *Hateful Symbols or Hateful People ? Predictive Features for Hate Speech Detection on Twitter*. Proceedings of NAACL-HLT 2016 Student Research Workshop.
- [6] Pavlopoulos, J., Malakasiotis, P. & Androutsopoulos, I. (2017). *Deep Learning for User Comment Moderation*. Proceedings of the First Workshop on Abusive Language Online, ACL 2017.
- [7] Zhang, X., Zhao, J. & LeCun, Y. (2018). *Character-level Convolutional Networks for Text Classification*. arXiv :1509.01626.
- [8] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019). *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019, pp. 4171–4186. <https://arxiv.org/abs/1810.04805>
- [9] Mozafari, M., Farahbakhsh, R. & Crespi, N. (2020). *A BERT-Based Transfer Learning Approach for Hate Speech Detection in Online Social Media*. Complex Networks and Their Applications VIII, Springer.
- [10] Hanu, L. & Unitary Team (2020). *Detoxify*. <https://github.com/unitaryai/detoxify>
- [11] Jigsaw / Google (2018). *Toxic Comment Classification Challenge*. Kaggle. <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
- [12] Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
- [13] Abadi, M. et al. (2015). *TensorFlow : Large-Scale Machine Learning on Heterogeneous Systems*. <https://www.tensorflow.org>